ver19 最終仕様書（OLED cavity / GPVM / phase3-BT proxy / strict-EML）

0. 目的とスコープ

本仕様書は、ver19 の最終的な実装を対象として、以下を包括的に記述する。

- `run_rgb_full_eval.sh` が呼び出す各プログラムの概要、入出力、アルゴリズム
- `common/` および `gpvm/` 以下の共通関数の仕様（数式・規約・参照面）
- GPVM 論文の式（特に Eq.(3), Eq.(26)–(30)）との対比
- phase3p1（policyB）最適化における BT proxy（rewrap）モデルと strict-EML モデルの違い
- TMM（transfer matrix method）と scattering matrix（S 行列）法の違い、および数値安定化手法
- 実装上の落とし穴（参照面、TM 符号規約、固定ファイル名上書き、cache、overflow）とその対策

1. リポジトリ構造（重要ファイル）

ver19 は、ルート直下の実行スクリプト群（`run_*.py`）と、サブディレクトリ実体群、および共通ライブラリで構成される。

■(A) ルート直下（entrypoint / wrapper を含む）

- `run_rgb_full_eval.sh`（RGB 一括実行ドライバ）
- `run_phase3_opt_then_gpvm_eml_profile.py`（WRITER：最適化＋ proxy/strict の出力）
- `run_gpvm_K_lambda_u0.py`（U0_MAIN：wrapper）
- `run_gpvm_K_lambda_u0_A_vs_B.py`（U0_AVSB：wrapper）
- `run_gpvm_K_lambda_kpar_map_strict.py`（HEATMAP：wrapper）
- `run_bottom_metal_PEC_proxy.py`（BOTTOM_METAL：k 依存性＋ Lcav profile）

ルート直下の `run_gpvm_...` は、実体スクリプトをサブディレクトリから import して `main()` を呼ぶ wrapper として定義される。これにより「実体側の BASE（repo root）解決」を安定化し、`--best-json` の相対パス解決の不整合を回避する。

■(B) 実体スクリプト（run_rgb_full_eval.sh の対象）

- `gpvm_k_lambda_u0/run_gpvm_K_lambda_u0.py`（strict-EML, u=0, full pol）
- `gpvm_k_lambda_u0/run_gpvm_K_lambda_u0_A_vs_B.py`（BT proxy vs strict 比較）
- `gpvm_K_lambda_kpar_map_strict/run_gpvm_K_lambda_kpar_map_strict.py`（strict-EML

heatmap）

■(C) **共通ライブラリ**

- `common/units.py`：単位変換（nm↔m 等）
- `common/oled_cavity_phase3p1_policyB.py`：phase3p1 の base と最適化
- `common/tmm_rewrap_utils_policyB.py`：BT 端反射（scattering 行列）＋ rewrap/proxy（Eq.(54)/(66)）
- `gpvm/*.py`：GPVM コア（Fresnel, kz, 行列, source-plane, Eq.(26)–(30) 等）

2. run_rgb_full_eval.sh：全体ワークフロー仕様

`run_rgb_full_eval.sh` は、RGB の 3 波長（既定：R=620nm, G=530nm, B=460nm）について、同一手順を繰り返し、出力を色別ディレクトリへ分離して生成する。

1. WRITER：最適化＋ best_geometry.json 作成＋ proxy/strict 出力
2. U0_MAIN：strict-EML の $K(\lambda, u = 0)$（成分分解つき）を出力
3. U0_AVSB：BT proxy と strict-EML の $K(\lambda, u = 0)$ を比較（検証図）
4. HEATMAP：strict-EML の $K(\lambda, k_{\|})$ heatmap（成分分解 ＋ 領域境界線）
5. BOTTOM_METAL：bottom metal (PEC proxy) 条件で $|E|^2$ の Lcav profile と、境界 $|E(0)|$ の k 依存性を出力

■(2.1) **上書き回避（色別ディレクトリ）**　WRITER は、`--best-json` の親ディレクトリに固定名の `.npy/.png` を出力するため、RGB で同一ディレクトリを共有すると上書きが起こる。ver19 では以下の構造を推奨・採用する：

`out_gpvm_step2/R/best_geometry.json`,　`out_gpvm_step2/G/best_geometry.json`,　`out_gpvm_step2/B/best_ge`

これにより、WRITER の固定名出力（proxy/field/strict など）も自動的に色別分離される。

■(2.2) cache **衝突回避（heatmap）**　heatmap は `*.npy` cache を `--outdir` に生成する。色ごとに `--outdir` を分けないと cache を誤読するため、RGB で `gpvm_K_lambda_kpar_map_strict/out_R`, `/out_G`, `/out_B` を分離する。

3. 物理モデルの二系統：phase3-BT proxy と strict-EML

ver19 の設計の核は、「phase3p1 最適化が見ている proxy（BT 境界＋ rewrap）モデル」と、「物理スタックに基づく strict-EML GPVM モデル」を明確に分離して扱うことである。

3.1 phase3-BT proxy（rewrap / objective）

phase3p1 の最適化は、EML 内部の電場そのものではなく、terminal 反射係数と光学座標（optical coordinate）を用いた Fabry–Perot proxy を用いる。

■(a) optical coordinate と terminal reflections　INTERNAL_ORDER（Lcav の内部層順）を

$$\texttt{INTERNAL_ORDER} = [\mathrm{pHTL}, \mathrm{Rprime}, \mathrm{HTL}, \mathrm{EBL}, \mathrm{EML}, \mathrm{ETL}]$$

とする。物理座標 z（nm）に対し、phase3 では層ごとの実屈折率を用いた optical coordinate

$$z_{\mathrm{opt}}(z) = \sum_{j:\ \mathrm{left} \to z} \mathrm{Re}(\tilde{n}_j)\, \Delta z_j$$

を定義し、Lcav の光学長

$$L_{BT} = \sum_{j \in \mathrm{INTERNAL_ORDER}} \mathrm{Re}(\tilde{n}_j)\, d_j$$

を用いる。Bottom/Top terminal reflection は、`common/tmm_rewrap_utils_policyB.py` で定義されるスタックに対して、

$$r_B(\lambda, u, \mathrm{pol}) \quad \text{at pHTL|ITO seen from pHTL,}$$
$$r_T(\lambda, u, \mathrm{pol}) \quad \text{at ETL|cathode1 seen from ETL}$$

として求める。

■(b) bottom phase policy（ϕ_B）　phase3-BT proxy は bottom 反射の位相をポリシーで制御する。既定では

$$r_B^{\mathrm{used}}(\lambda, u, \mathrm{pol}) = -|r_B^{\mathrm{raw}}(\lambda, u, \mathrm{pol})|$$

（$\phi_B = \pi$ を強制）とする。これは「最適化の目的関数と図の再現性」のための規約であり、物理 stack の位相とは一致しない場合がある。

■(c) rewrap 反射（ru/rd）と proxy $E2_{\mathrm{proxy}}(z)$　phase3 では、位置 z における rewrap 反射を

$$r_u(z) = r_B^{\mathrm{used}} \exp(i\, 2k_0\, z_{\mathrm{opt}}(z))\,, \qquad r_d(z) = r_T \exp(i\, 2k_0\, (L_{BT} - z_{\mathrm{opt}}(z)))\,,$$
$$k_0 = \frac{2\pi}{\lambda}$$

と定義する。Fabry–Perot 分母は

$$D(z) = 1 - r_u(z) r_d(z)$$

であり、phase3 の未正規化 proxy は

$$E2_{\mathrm{proxy}}(z) = \frac{|1 + r_u(z)|^2\, |1 + r_d(z)|^2}{|D(z)|^2} = \frac{|1 + r_u(z)|^2\, |1 + r_d(z)|^2}{|1 - r_u(z) r_d(z)|^2}. \tag{1}$$

この $E2_{\mathrm{proxy}}(z)$ は「最適化で用いる評価量」であり、strict-EML GPVM の $|E|^2$ とは別物である。

3.2 strict-EML GPVM（物理 stack）

strict-EML は、実際の stack（EML 界面で左/右に分割）から reflection coefficient を抽出し、GPVM 論文の式に従って K や field を計算する。

■(a) kinematics：u と k_z　参照屈折率を EML の屈折率 $\tilde{n}_e$ とし、面内波数を

$$k_{\parallel} = k_0\,\tilde{n}_e\,u$$

で表す。各層 j の縦波数は

$$k_{z,j} = k_0\sqrt{\tilde{n}_j^2 - \tilde{n}_e^2 u^2}$$

で定義し、枝は

$$\mathrm{Im}(k_{z,j}) \geq 0$$

となるよう選択する（エバネッセント波が界面から離れる方向に減衰）。

■(b) EML 界面参照の left/right stack と rA/rB　EML 層内の source plane（発光面）を EML 左端から距離 z_{ex} に置く。left stack を「EML から見て左側」、right stack を「EML から見て右側」とし、TE/TM 別に transfer matrix から反射係数を抽出して

$$r_A(\lambda, u, \mathrm{pol}), \quad r_B(\lambda, u, \mathrm{pol})$$

を得る。これらは後述の GPVM Eq.(26)–(30) に入る。

4. 数値計算法：TMM と scattering matrix の違い、および安定化

4.1 TMM（transfer matrix method）

2×2 transfer matrix を用い、右側振幅から左側振幅への写像

$$\begin{pmatrix} E_L^+ \\ E_L^- \end{pmatrix} = M \begin{pmatrix} E_R^+ \\ E_R^- \end{pmatrix}$$

を構成する。多層膜では界面行列 $I^{j/(j+1)}$ と層内伝搬行列 $L^j(d_j)$ を掛け合わせて

$$M = I^{0/1} L^1(d_1) I^{1/2} L^2(d_2) \cdots I^{N-2/(N-1)}$$

を得る。

■(a) 界面行列（GPVM Eq.(2)）　GPVM 論文の Eq.(2) に従い、界面行列を

$$I^{j/(j+1)} = \frac{1}{t_{j/(j+1)}} \begin{pmatrix} 1 & r_{j/(j+1)} \\ r_{j/(j+1)} & 1 \end{pmatrix}$$

とする（r, t は GPVM Eq.(3) の Fresnel 係数）。

■(b) **層内伝搬行列（GPVM Eq.(1)）**　理想的には

$$L^j(d) = \begin{pmatrix} e^{-ik_{z,j}d} & 0 \\ 0 & e^{+ik_{z,j}d} \end{pmatrix}$$

だが、$\mathrm{Im}(k_{z,j}) \geq 0$ のとき $e^{-ik_{z,j}d}$ は $e^{+\mathrm{Im}(k_{z,j})d}$ を含み発散しうる。

4.2 TMM の数値不安定性（overflow の原因）

エバネッセント波では $\mathrm{Im}(k_{z,j}) > 0$ となり、transfer matrix の生の伝搬因子

$$e^{-ik_{z,j}d} = e^{-i\mathrm{Re}(k_{z,j})d}\, e^{+\mathrm{Im}(k_{z,j})d}$$

が指数的に増大する。深いスタックや短波長ではこれが浮動小数の上限を超え、

$$\exp(+\mathrm{Im}(k_{z,j})d) \to \infty$$

となって overflow を起こし、以後の行列積が NaN/Inf に汚染される。

4.3 ver19 の TMM 安定化（スカラー正規化）

ver19 の `gpvm/matrices.py` と `gpvm/system_matrix.py` は、TMM を維持しつつ以下の安定化を導入する。

■(a) **層行列のスカラー因子除去**　反射係数抽出は行列要素比（例：$-M_{01}/M_{00}$）で行うため、全体スカラー倍は r, t を変えない。そこで

$$L^j(d) \mapsto e^{-\mathrm{Im}(k_{z,j})d}\, L^j(d)$$

として、発散する $e^{+\mathrm{Im}(k_{z,j})d}$ を除去する。実装では

$$L^j(d) = \begin{pmatrix} e^{-i\mathrm{Re}(k_{z,j})d} & 0 \\ 0 & e^{+i\mathrm{Re}(k_{z,j})d}\, e^{-2\mathrm{Im}(k_{z,j})d} \end{pmatrix}$$

を返し、$|L_{22}| \leq 1$ を保証する。

■(b) **行列積の** renorm　行列積の各ステップで

$$M \leftarrow \frac{M}{\max(|M_{ab}|)}$$

（スカラー）正規化を行い、overflow を防ぐ。これも比で抽出される r, t を変えない。

4.4 scattering matrix（S 行列）法（terminal reflection の安定計算）

transfer matrix の代わりに scattering matrix を用いると、エバネッセント波で指数発散する項を自然に避けられる。ver19 では BT 端反射を `common/tmm_rewrap_utils_policyB.py` 内で S 行列カスケードにより計算する（PyMoosh を本番依存から外し、テスト参照に限定する方針）。

■(a) **界面 S 行列**　媒質 1（左）と媒質 2（右）の「admittance」b を

$$b = \begin{cases} k_z & (\mathrm{TE}) \\ k_z/\varepsilon & (\mathrm{TM}) \end{cases}, \qquad \varepsilon = \tilde{n}^2$$

とする（これは PyMoosh の `coefficient_S` と整合する規約）。界面 scattering 行列は

$$S_{\mathrm{int}}(b_1, b_2) = \frac{1}{b_1 + b_2} \begin{pmatrix} b_1 - b_2 & 2b_2 \\ 2b_1 & b_2 - b_1 \end{pmatrix}.$$

■(b) **層 S 行列**　厚さ d の均一層の伝搬因子を

$$t = e^{ik_z d}$$

とし、

$$S_{\mathrm{layer}}(t) = \begin{pmatrix} 0 & t \\ t & 0 \end{pmatrix}$$

とする。枝を $\mathrm{Im}(k_z) \geq 0$ としているため、エバネッセント波では

$$|t| = |e^{ik_z d}| = e^{-\mathrm{Im}(k_z)d} \leq 1$$

となり発散しない（TMM の $e^{+\mathrm{Im}d}$ と対照的）。

■(c) **カスケード合成**　2 つの散乱行列 A と B を直列に接続した合成 $S = A \circ B$ は

$$t = \frac{1}{1 - B_{00}A_{11}},$$

$$S_{00} = A_{00} + A_{01}B_{00}A_{10}t, \quad S_{01} = A_{01}B_{01}t,$$

$$S_{10} = B_{10}A_{10}t, \quad S_{11} = B_{11} + A_{11}B_{01}B_{10}t.$$

全体散乱行列 S の反射係数は

$$r = S_{00}$$

として得る（入射側から見た反射）。

5. Fresnel 係数：fresnel.pdf と GPVM Eq.(3) の対比

5.1 角度表示（fresnel.pdf）

入射側 i、透過側 t に対して

$$r_s = \frac{\tilde{n}_i \cos\theta_i - \tilde{n}_t \cos\theta_t}{\tilde{n}_i \cos\theta_i + \tilde{n}_t \cos\theta_t}, \qquad t_s = \frac{2\tilde{n}_i \cos\theta_i}{\tilde{n}_i \cos\theta_i + \tilde{n}_t \cos\theta_t},$$

$$r_p = \frac{\tilde{n}_t \cos\theta_i - \tilde{n}_i \cos\theta_t}{\tilde{n}_t \cos\theta_i + \tilde{n}_i \cos\theta_t}, \qquad t_p = \frac{2\tilde{n}_i \cos\theta_i}{\tilde{n}_t \cos\theta_i + \tilde{n}_i \cos\theta_t}.$$

5.2 u 表示（GPVM）と等価性

面内波数保存より

$$k_{\parallel} = k_0 \tilde{n}_e u = k_0 \tilde{n}_j \sin\theta_j$$

なので

$$\tilde{n}_j \cos\theta_j = \sqrt{\tilde{n}_j^2 - \tilde{n}_e^2 u^2}.$$

これを上式に代入することで GPVM Eq.(3) の u 表示が得られる。ver19 の `gpvm/fresnel.py` は GPVM Eq.(3) を実装し、特に TM 反射係数に追加のマイナス符号を持つ規約を採用する：

$$r_{\mathrm{TM}}^{\mathrm{GPVM}} = -\frac{k_{z,j}/\tilde{n}_j^2 - k_{z,k}/\tilde{n}_k^2}{k_{z,j}/\tilde{n}_j^2 + k_{z,k}/\tilde{n}_k^2}, \qquad t_{\mathrm{TM}}^{\mathrm{GPVM}} = \frac{2k_{z,j}}{k_{z,j}(\tilde{n}_k/\tilde{n}_j) + k_{z,k}(\tilde{n}_j/\tilde{n}_k)}.$$

一方、rewrap（PyMoosh 互換）側では TM の界面反射に $b = k_z/\varepsilon$ を用いる規約（追加マイナス無し）を採用する。従って TM の符号規約は「どの行列定義と組み合わせるか」に依存し、混在させないことが重要である。

6. GPVM コア式：Eq.(26)–(30)（ver19: gpvm/eqs_gpvm.py）

ver19 では Eq.(26)–(30) の計算を `gpvm/eqs_gpvm.py` に共通化した。全ての strict-EML 系スクリプト（U0_MAIN, HEATMAP）はこの実装を参照する。さらに U0_AVSB は Eq.(26)+Eq.(30) を wrapper として呼ぶ（重複排除）。

6.1 共通分母（Fabry–Perot）

$$D = 1 - r_A r_B e^{i2k_z d}$$

を共通に用いる（`_D()`）。

6.2 TE 水平双極子（Eq.(26)）

$$P_e^{\mathrm{TE},h} = \frac{3}{16\pi}\left[(1-R_B)\left|\frac{1+r_A e^{i2k_z z_{ex}}}{D}\right|^2 + (1-R_A)\left|\frac{1+r_B e^{i2k_z(d-z_{ex})}}{D}\right|^2\right],$$

$$R_A = |r_A|^2, \quad R_B = |r_B|^2.$$

6.3 TM 水平双極子（Eq.(27)）

$$P_e^{\mathrm{TM},h} = \frac{3}{16\pi}(1-u^2)\left[(1-R_B)\left|\frac{1+r_A e^{i2k_z z_{ex}}}{D}\right|^2 + (1-R_A)\left|\frac{1+r_B e^{i2k_z(d-z_{ex})}}{D}\right|^2\right].$$

6.4 TM 垂直双極子（Eq.(28)）

Eq.(28) は分子の符号が異なる（重要）：

$$P_e^{\mathrm{TM},v} = \frac{3}{8\pi}u^2\left[(1-R_B)\left|\frac{1-r_A e^{i2k_z z_{ex}}}{D}\right|^2 + (1-R_A)\left|\frac{1-r_B e^{i2k_z(d-z_{ex})}}{D}\right|^2\right].$$

6.5 K 変換（Eq.(30)）

$$K(u) = \frac{\pi}{1-u^2}P_e(u).$$

実装では $1-u^2$ のゼロ割を避けるため $\max(10^{-30}, 1-u^2)$ を用いる。

6.6 等方平均（コード規約）

ver19 の出力は以下で定義される：

$$K_{\mathrm{TE_only}} = \frac{2}{3}K_{\mathrm{TE},h}, \quad K_{\mathrm{TM_h_only}} = \frac{2}{3}K_{\mathrm{TM},h}, \quad K_{\mathrm{TM_v_only}} = \frac{1}{3}K_{\mathrm{TM},v},$$

$$K_{\mathrm{iso}} = K_{\mathrm{TE_only}} + K_{\mathrm{TM_h_only}} + K_{\mathrm{TM_v_only}}.$$

7. 各スクリプト仕様（run_rgb_full_eval.sh が使用）

以下では、WRITER / U0_MAIN / U0_AVSB / HEATMAP / BOTTOM_METAL の仕様を記述する。

7.1 WRITER：run_phase3_opt_then_gpvm_eml_profile.py

■目的　phase3p1（policyB）の既存最適化経路を用いて、指定波長 λ に対する best geometry（`best_geometry.json`）を生成し、次を出力する。

- phase3 の目的関数で使う未正規化 proxy $E2_{\mathrm{proxy}}(z)$ の Lcav プロファイル（推奨出力）
- strict-EML GPVM による EML 内 profile（診断用）
- （任意）phase3-BT の source-plane field を用いた EML 内 profile（診断用）

重要：最適化の正当性検証には proxy が本体であり、strict-EML の $|E|^2$ と一致する必要はない。

■CLI input

- `--lambda-nm`：最適化・評価波長（nm、既定 650）
- `--best-json`：出力する best_geometry.json のパス（相対はスクリプト位置基準）

■主要 output（–best-json の親ディレクトリへ出力）

- best_geometry.json（厚み、最適化パラメータ、lambda_nm、LBT、z_ex 等）
- phase3 proxy（Lcav 全域）：
  - z_lcav_nm_phase3BT.npy
  - zopt_lcav_nm_phase3BT.npy
  - E2_proxy_lcav_phase3BT_raw.npy（未正規化：目的関数そのもの）
  - E2_proxy_lcav_phase3BT_norm.npy（可視化用）
  - gpvm_lcav_proxy_phase3BT.png（EML 境界＋中心点線入り）
- strict-EML EML profile：
  - z_eml_nm.npy, I_TE_h_strictEML.npy 等
  - gpvm_eml_profiles_strictEML.png

■アルゴリズム（要点）

1. common/oled_cavity_phase3p1_policyB.py の optimize_etl_then_s() を呼び、ETL 厚 etl_best とスケール s_best を探索する。
2. 最適化の目的関数は、proxy $E2_{\mathrm{proxy}}(z)$ のピーク位置 z_{peak} と EML 中心 z_{center} の距離

$$\mathrm{score} = |z_{\mathrm{peak}} - z_{\mathrm{center}}|$$

   （ピーク誤差）を最小化する。
3. terminal reflections を common.tmm_rewrap_utils_policyB.terminal_reflections_BT() で計算し、(**??**) に従い Lcav 全域の $E2_{\mathrm{proxy}}$ を出力する。

■（重要）発見されたバグと修正　過去に、BT terminal 参照の r_B, r_T をそのまま source-plane solver に渡してしまい、field-like profile が破綻するバグがあった。ver19 では、

$$r_A^{(\mathrm{src})} = r_B^{\mathrm{used}} e^{i2k_0 z_{ex,\mathrm{opt}}}, \qquad r_B^{(\mathrm{src})} = r_T e^{i2k_0 (L_{BT} - z_{ex,\mathrm{opt}})}$$

として source-plane 参照に変換してから solve_source_plane_fields() に渡す（proxy 出力自体はこの変換を必要としないが、診断 field の整合性のため実装した）。

7.2 U0_MAIN：gpvm_k_lambda_u0/run_gpvm_K_lambda_u0.py

■目的　strict-EML（物理 stack）に基づき、$u = 0$ の $K(\lambda)$ を TE/TM × (h,v) 成分分解で計算し、*.npy と図を出力する（定量本体）。

■CLI input

- --best-json：geometry JSON（既定：./out_gpvm_step2/best_geometry.json）
- --outdir：出力ディレクトリ

- `--tag`：出力ファイル名サフィックス（_R/_G/_B 等）
- 波長グリッド:`--lam-center-nm`, `--lam-span-nm`, `--lam-min-nm`, `--lam-max-nm`, `--n-lam`

■Output（例）

- `lam_nm_{tag}.npy`
- `K_TE_h_{tag}.npy`, `K_TM_h_{tag}.npy`, `K_TM_v_{tag}.npy`
- `K_iso_{tag}.npy` 等
- `gpvm_K_lambda_u0_components_strict_{tag}.png`

■アルゴリズム（要点）

1. `common.oled_cavity_phase3p1_policyB.build_current_base()` から n_0, d_0 を取得し、ITO を PEC 近似金属に置換：
$$\tilde{n}_{\mathrm{ITO}} = 0.14 + 2000i.$$
2. EML 屈折率は lossless として $\tilde{n}_e = \mathrm{Re}(\tilde{n}_{\mathrm{EML}})$ を用いる。
3. EML 界面で left/right stack を構築し、各 λ で TE/TM 別に transfer matrix を計算して r_A, r_B を抽出する。
4. Eq.(26)–(30) を `gpvm/eqs_gpvm.py` で評価し、成分および K_{iso} を出力する。

7.3 U0_AVSB：gpvm_k_lambda_u0/run_gpvm_K_lambda_u0_A_vs_B.py

■目的　同一 geometry に対して

$$\text{A: phase3-BT proxy} \quad \text{vs} \quad \text{B: strict-EML}$$

の $K(\lambda, u = 0)$ を比較し、ズレを可視化する（検証用）。

■成分の扱い　比較目的のため TE × h のみを計算し、等方平均を

$$K_{\mathrm{iso}} \approx \frac{2}{3} K_{\mathrm{TE},h}$$

として扱う（TM 成分を含めた全成分比較は本仕様外だが拡張可能）。

■なぜ Eq.(26) と Eq.(30) を合成するのか（設計意図）　u=0 固定では Eq.(30) は $K = \pi P_e$ の定数倍であり、中間量 P_e を保存する必要が薄い。したがって過去実装では Eq.(26) と Eq.(30) を合成した関数として記述されていた。ver19 では共通化のため `Pe_TE_h_eq26` と `K_from_Pe_eq30` の wrapper 呼び出しに統一した。

■Output

- `gpvm_K_lambda_u0_A_vs_B_{tag}.png`
- （必要最小の）`lam_nm`, `K_A`, `K_B` の `.npy`

7.4 HEATMAP：gpvm_K_lambda_kpar_map_strict/run_gpvm_K_lambda_kpar_map_strict.py

■目的　strict-EML GPVM により、$K(\lambda, k_\parallel)$（2 次元）を TE/TM × (h,v) 成分分解で計算し、heatmap と検証図を生成する。

■CLI input

- `--best-json`, `--outdir`, `--tag`
- 波長グリッド:`--lam-center-nm`, `--lam-span-nm`, `--lam-min-nm`, `--lam-max-nm`, `--n-lam`

■k_parallel グリッド　$u_{\max} \approx 0.99$ を用い、最短波長での k_0 から

$$k_{\parallel,\max} = u_{\max}\,\mathrm{Re}(\tilde{n}_e)\,k_0(\lambda_{\min})$$

を定め、$k_\parallel \in [0, k_{\parallel,\max}]$ を離散化する（実装では $\mu\mathrm{m}^{-1}$ 表示も出力）。

■Output

- cache（tag なし）：`lam_nm.npy`, `kpar_grid_um.npy`, `K_*_map.npy`, `K_*_u0.npy`
- figures（tag 付き）：linear/log、軸入れ替え版、境界線オーバレイ、k//=0 チェック図

■数値安定化　heatmap は evanescent を含みやすく、TMM の overflow が起きやすい。ver19 では §4.3 のスカラー正規化により、exp overflow の連鎖を抑止する。

7.5 BOTTOM_METAL：run_bottom_metal_PEC_proxy.py

■目的　bottom 境界を半無限金属（PEC proxy）で近似し、(i) 境界 $|E(0)|$ の k 依存性、(ii) 代表 k（既定 2000）での Lcav 全域 $|E|^2$ profile を出力する。

■CLI input

- `--best-json`, `--outdir`, `--tag`
- `--xmax-nm`：プロットの x 軸最大を固定（RGB 整列用）

■モデル　bottom 半無限媒質を

$$\tilde{n}_{\mathrm{metal}} = 0.14 + ik$$

として置換し、k を掃引する（TE × h, $u = 0$）。

■出力

- `need_k_for_E0_small_{tag}.csv`（k と ratio）
- `need_k_for_E0_small_{tag}.png`
- `gpvm_lcav_profile_bottomMetal_k2000_{tag}.png`

8. 共通関数仕様（common/）

8.1 common/units.py

依存ゼロの純関数として、nm↔m 等の変換を提供する。配列・スカラー両対応であり、スカラーは `float` を返す。

$$\mathrm{nm_to_m}(x) = x \times 10^{-9}, \qquad \mathrm{m_to_nm}(x) = x \times 10^{9}.$$

8.2 common/oled_cavity_phase3p1_policyB.py

phase3p1 のベースモデル（屈折率と初期厚）と最適化ルーチンを提供する。

- `build_current_base()`：n_0, d_0 を返す（屈折率は基本的に固定値）
- `optimize_etl_then_s(...)`：ETL 厚とスケールを探索し、proxy ピークが EML 中心に来る候補を返す
- `INTERNAL_ORDER`：Lcav 内部層の順序

最適化が minimization する量は（proxy を用いた）ピーク位置誤差であり、境界 $|E(0)|$ を小さくすることは目的関数に含まれない。

8.3 common/tmm_rewrap_utils_policyB.py

BT 端反射の評価と、phase3 proxy（rewrap）の計算を提供する。ver19 の本番経路は PyMoosh に依存せず、S 行列カスケードで terminal reflection を計算する（§4.4）。

- `terminal_reflections_BT(...)`：$(r_B^{used}, r_T, r_B^{raw})$ を返す（配列）
- `stack_reflection_smatrix_spectrum(...)`：任意スタックの $r(\lambda)$ を返す（安定）
- `effective_ru_rd_at_S_from_BT(...)`：r_u, r_d の構成（rewrap）
- `green_terms_from_ru_rd(...)`：proxy 式（(**??**) 等）を返す

9. GPVM コア（gpvm/）

9.1 gpvm/kz.py

$$k_z = k_0\sqrt{\tilde{n}^2 - \tilde{n}_e^2 u^2}$$

を計算し、枝を $\mathrm{Im}(k_z) \geq 0$ に固定する（エバネッセント減衰）。

9.2 gpvm/fresnel.py（GPVM Eq.(3)）

`fresnel_rt()` は TE/TM の r, t を返す。TM 反射には論文規約に合わせた追加マイナスが入る（§5.2）。

9.3 gpvm/matrices.py（Eq.(1),(2)）

- `layer_matrix()`：§4.3(a) のスカラー因子除去を実装（overflow 回避）
- `interface_matrix()`：$I = (1/t)\begin{pmatrix} 1 & r \\ r & 1 \end{pmatrix}$

9.4 gpvm/system_matrix.py

- `stack_transfer_matrix()`：多層 transfer matrix 構築（renorm あり）
- `rt_from_transfer_matrix()`：左入射（$E_R^- = 0$）の (r, t) を抽出
- `rt_incident_from_right()`：右入射の (r, t) を抽出（Eq.(12) の S^A 用）
- `build_system_matrices_SA_SB()`：GPVM Eq.(6),(7) の S^A, S^B 構築
- `eq12_rt_from_SA_SB()`：Eq.(12) の (r_A, t_A, r_B, t_B) を抽出

9.5 gpvm/source_terms.py と gpvm/source_plane.py

`source_terms()` は TE/TM, h/v に応じた source term（Eq.(5) に相当する係数）を返し、`solve_source_plane_fields()` は source-plane 参照の r_A, r_B を用いて source plane 両側の進行波振幅 $(E_a^{\pm}, E_b^{\pm})$ を解く。参照面を誤ると field-like profile が破綻するため、terminal 反射を投入する際は必ず source-plane 参照へ変換する。

10. テスト方針（概要）

ver19 では「run_rgb_full_eval.sh が用いるスクリプトと gpvm/common の共通ライブラリ」を検証対象とし、可能な限り PyMoosh（`PyMoosh-stable`）を参照（ref）として比較する。

- Fresnel（rewrap 規約） vs PyMoosh interface
- BT terminal reflections（rb_raw/rt） vs PyMoosh coefficient_S
- proxy $E2_{\text{proxy}}(z)$ の自己一致（最適化内部と WRITER 出力）
- TMM stack r vs PyMoosh（strict 側）
- bottom metal の k 依存性（単調性＋閾値）

11. 運用上の注意（事故の典型と対策）

- **固定名出力の上書き**：WRITER は `--best-json` の親に固定名を出す。RGB は必ず色別ディレクトリへ。
- **cache 衝突**：heatmap の `*.npy` は `--outdir` に固定名で作られる。色別 outdir 分離必須。
- **参照面の混同**：terminal 反射と source-plane 反射を混ぜない。solver に入れる r_A, r_B は source-plane 参照。

- **TM 符号規約**：GPVM（`gpvm/fresnel.py`）と rewrap（PyMoosh 互換）の TM 規約は異なる。用途ごとに一貫させる。
- **overflow/NaN**：evanescent 領域で発散しやすい。TMM 安定化（スカラー除去＋ renorm）と、端反射は S 行列法を使う。
- **SSH 切断**：`source` でシェルに `set -e` を持ち込むと、エラーでログインシェルが exit し SSH が切れたように見える。`bash run_rgb_full_eval.sh` で実行する。

12. まとめ

ver19 は、phase3 の最適化が依拠する BT proxy（rewrap / proxy）と、strict-EML GPVM（物理 stack）を分離し、両者の関係（比較）を U0_AVSB で監視できる構成とした。数値安定化として、(i) TMM 側のスカラー正規化、(ii) BT 端反射の S 行列カスケード実装、(iii) 波長グリッドの可変化、(iv) wrapper 化による BASE 解決の安定化、を導入した。これにより RGB 一括評価（`run_rgb_full_eval.sh`）が再現性高く実行でき、最適化（proxy）と物理評価（strict）を同一リポジトリ内で明確に比較・検証できる。